Journal of Information Security and Applications 76 (2023) 103543

Contents lists available at ScienceDirect

Journal of Information Security and Applications

journal homepage: www.elsevier.com/locate/jisa

Modeling and physical attack resistant authentication protocol with double PUFs

Shen Hou [a,*], Yanzhou Ma [a], Ding Deng [b], Zhenyu Wang [c], Guolei Ren [d]

[a] Department of Reconnaissance Intelligence, Information Engineering University, Luoyang, China
[b] College of Electronic Science and Technology, National University of Defense Technology, Changsha, China
[c] College of Computer, National University of Defense Technology, Changsha, China
[d] Centre of Teaching and Research Support, National University of Defense Technology, Changsha, China

ARTICLE INFO

Keywords:
Physical unclonable function
Authentication protocol
LFSR
Machine learning attacks
Physical attacks

ABSTRACT

As new lightweight hardware security primitive, the physical unclonable function (PUF) is used in various hardware-based secure applications like keys generation and device authentication. But PUFs are vulnerable to modeling attacks using machine learning (ML) technology. To solve the problem, a dynamically configurable obfuscation unit for PUF using a dynamic linear feedback shift register (DLFSR) is proposed. The structure of this obfuscation unit can change with the clock cycle running, making the PUF difficult to model. Using the obfuscation unit, a dual-PUF-based mutual device authentication protocol for low-cost IoT systems is proposed. New technologies like dual-stage and device-to-server authentication are used to limit the amount of data that an attacker can obtain. A random pattern-based mechanism is introduced to the protocol to enhance the robustness against physical attacks. A mathematical model of the proposed unit is presented and analyzed. Experimental results on FPGA show that the design is simpler, more reliable, and lightweight. It can effectively resist ML-based attacks and physical attacks.

1. Introduction

In freshly years, the rapid advancement of wireless, mobile, and low-power connection technologies has led to the proliferation of the Internet of Things (IoT). More and more IoT devices have been used in industrial and daily life scenarios. According to Gartner, the total amount of IoT appliances will come to 11.57 billion by 2022 [1]. This causes many security issues on IoT devices and low-cost network communication. In traditional security mechanisms, cryptographic algorithms like AES and Hash are used to prevent these attacks. However, they are very difficult to be integrated into IoT systems because of the expensive computational, memory, and communication overhead. Therefore, developing lightweight security technologies for resource-constrained IoT and embedded devices is critical.

During the manufacturing of digital circuits, for some uncontrolled reasons, the actual parameters of the circuits may have slight deviations. These deviations are chip uniqueness and can be extracted to build a "fingerprint" of the chip. Such circuits are called physical unclonable functions (PUFs). PUF is a new lightweight hardware security primitive and needs no extra non-volatile memory (NVM) to store secret information. The working scheme of PUF can be defined as a mapping function as $r = P(c)$. The input c is called the challenge and the output r is called the response. For a PUF circuit, only when a challenge c is loaded, the corresponding response r is generated. The c and the corresponding r are called a challenge-response pair (CRP). According to the number of total CRPs, PUFs can be split into two categories: weak PUFs and strong PUFs [2]. Weak PUFs have a smaller set of CRPs and are usually used in secret key generating. Strong PUFs have an exponentially large CRPs set, which is very fitting for IoT appliances security authentication [3].

However, a major problem of many strong PUFs is that their simple structure brings circuit correlations between different CRPs. These correlations can be precisely numerically modeled by an attacker using ML algorithms. For example, Arbiter PUF (APUF) is a well-performance delay-based strong PUF design, which is used in many security authentication protocols. In 2013, Ruhrmair et al. showed that an attacker can train a model with 95% accuracy in 0.01 s and only needs 640 CRPs of a 64-bit APUF [4]. An attacker can easily collect a CRPs dataset by eavesdropping on the communication channel, compromising the authentication, and hacking into the system [5]. Besides that, Delvaux illustrated that new PUF-based authentication protocols research still has many problems. He has presented several ML and cryptanalysis attacking methods to compromise the PolyPUF protocol of Konigsmark

* Corresponding author.
E-mail address: houshen@outlook.my (S. Hou).

https://doi.org/10.1016/j.jisa.2023.103543

Available online 30 June 2023

et al. [6], the OBPUF protocol of Gao et al. [7], the RPUF protocol of Ye et al. [8], the LHS-PUF protocol of Idriss and Bayoumi [9], and the PUF-FSM protocol of Gao et al. [10]. We recently proposed a lightweight, secure PUF-based IoT device mutual authentication using a device-side obfuscation unit to protect the PUF instances, and a protocol with two separate stages to further limit the leak of CRPs [11]. However, the two-stage protocol is vulnerable to physical attacks.

In this article, the improved obfuscation unit and physical attack resist protocol are proposed. The evaluating results show that even simple APUFs can effectively resist modeling and physical attacks when equipped with the proposed method. This is achieved by the following main contributions:

- The obfuscation unit is built with a configurable DLFSR with configurable parameters and a dynamic changing structure. The use of sequential logic increases the nonlinearity of the system greatly without more extra hardware overhead.
- The use of two PUF instances in two authentication stages brings robustness against man-in-the-middle and tampering attacks.
- A random pattern approach is used on the protocol level to resist physical invasive attacks.
- Security performances of the method are theoretically analyzed. It is fully implemented on FPGA using APUF instances and is evaluated with more rigorous security standards.

This paper is organized as follows: Section 2 introduces the background and related work; Section 3 describes the detailed implementation of the DLFSR-based obfuscation unit and the authentication protocol design; Section 4 analyzes the security performances; Section 5 shows a typical physical attack scenario and gives an improved protocol. Section 6 presents and analyzes the experiment results both in performance and security; Section 7 is the conclusion and future prospects of the article.

2. Background

In this section, the background information on APUF, modeling attack, and PUF-based authentication protocol are provided first. Then some related research is introduced.

2.1. Arbiter PUF and modeling attack

An APUF consists of a multi-stage multiplexer chain. The fundamental design and the delay model of APUF are displayed in Fig. 1. A 1-bit APUF instance has two delay paths called the top path and the down path. They have the same input pulse but the connection way in each stage is dynamically changing. The connection way is controlled by an input signal called a challenge. The route and length of the two delay paths are exactly the same, but this cannot be achieved in the manufacturing process. The time delay values of the two paths will be slightly different because of the manufacturing process deviation. An arbiter circuit at the end of the chain determines which signal pulse arrives first. The two arrival cases represent 0 and 1 respectively. The design of APUF is uncomplicated and lightweight, and the physical implementation of APUF is relatively simple. So, APUF has a broad application prospect as a key hardware security enhancement for IoT appliances.

The delay difference between the top and bottom paths of an APUF instance is the sum of all stages. According to Rührmair, the total delay difference between the top and bottom paths of an n-stage APUF (the length of input challenge is n) is:

$$\Delta = \sum_{i=1}^{n} w_i \varphi_i = \boldsymbol{w} \cdot \boldsymbol{\varphi}, \tag{1}$$

where $i = 0, 1, 2, \ldots, n$. $\boldsymbol{\varphi}$ is determined by the input challenge and named the feature vector. $\boldsymbol{w}$ is named the weight vector of the APUF instance. So, the numerical model of the APUF is only a simple linear function.

Generally speaking, modeling attacks are used to compromise strong PUF designs. An adversary may try to get the logical detail and a certain number of true CRPs of the target strong PUF. With this information, the adversary can abstract a numerical model of the target PUF and trains it using a learning algorithm. As the training data increases and the algorithm improves, the model will become more accurate. Then this model can be used to predict unseen CRPs of the target PUF precisely. To compromise an APUF instance, attackers can obtain the model parameter $\boldsymbol{w}$ by simple ML classification algorithms.

2.2. Strong PUF-based authentication protocol

A typical application scenario of strong PUFs in IoT is the secure authentication of devices. Fig. 2 shows a basic strong PUF device authentication protocol, that each device in the network has an embedded APUF instance. The authentication protocol usually consists of two stages:

(1) *Enrollment stage*. In this stage, the designer or the manufacturer should gather a real CRPs set of each inner PUF instance of all the embedded devices. All the CRPs sets should be stored in a secure server or database. This process is called the enrollment stage. The CRPs are used to do authentication in the next stage.

(2) *Authentication stage*. In this stage, the user or the customer can use the stored CRPs data to authenticate devices. The basic process can deploy by simple information exchange between the secure server and a device i. The main steps are:

- `Step1`: The user chooses an unused CRP from the secure server by some random method and passes the challenge sector to the device.
- `Step2`: The function unit on the device receives the challenge vector and deploys it to the inner PUF unit. Then a response is generated by the PUF unit which is passed back.
- `Step3`: When the response is received, the server can make a comparison between the received response and the response sector of the original CRP. If the difference is larger than the threshold set up by the user, the user can consider the device compromised.

Compared with traditional authentication protocols with cryptographic primitives, PUF-based authentication protocols have a simpler architecture, lower hardware, and communication overhead, and are suitable for IoT applications. However, this basic version is not secure because the inner PUF instance is vulnerable to ML-based modeling attacks.

2.3. Secure enhancing works

To enhance the resistance to modeling attacks, many new designs were proposed in recent years. They can be roughly classed into three categories:

(1) Non-linear design method to increase the complexity between challenges and responses. New digital and analog features in IC design are abstracted as PUF designs. Representative designs are feed-forward APUF [12], non-linear current mirror PUF [13], switched-capacitor PUF [14], ReRAM PUF [15], etc. Multiple APUF instances are combined in a non-linear way to create a secure combination PUF. Representative designs are XOR APUF, Double APUF [16], MPUF [17], interpose PUF [18], etc. This design method is technically difficult and uncertain. Although reliable, stable, and non-linear electrical characteristics are widely present in silicon circuits, extracting the process deviation of these characteristics for PUF designing is hard to achieve. Moreover, a so-called good PUF design should be able to resist the current mainstream attacks. However, many PUF components are usually needed, which not only raises hardware overhead but also rapidly decreases

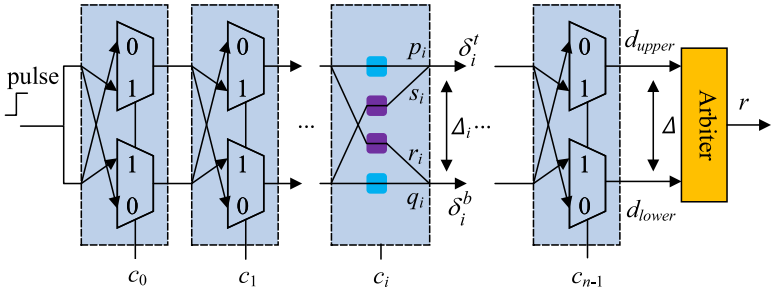

Fig. 1. Structure of an n-stage APUF instance.

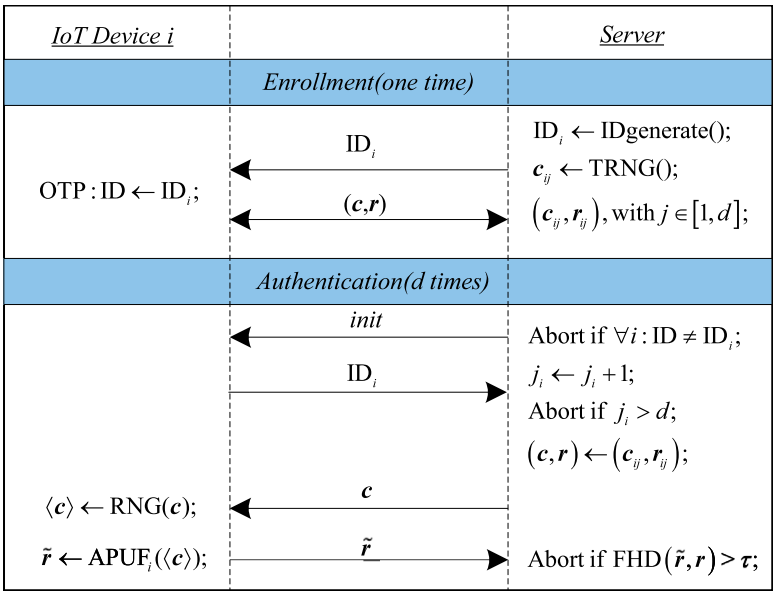

Fig. 2. The basic strong PUF authentication protocol.

the reliability of the PUF. Poor reliability decreases the availability of PUFs and leads to reliability-based modeling attacks [19].

(2) Non-linear obfuscation logic to hide the true PUF models. The obfuscation logic can be used on challenges, responses, or both. Not knowing the obfuscation method, attackers cannot obtain true CRPs. Representative designs are RPUF [8], PUF-FSM [10], etc. This design method is easily implemented and has lower R&D costs. However, advanced deep learning (DL) techniques make the modeling process easier and easier, and the sample set for training is decreased. Simple obfuscation methods can no longer defend against powerful ML and DL attacks [20]. The RPUF and OPUF were broken by the cryptoanalysis method. Additionally, obfuscation logic is vulnerable to physical attacks. Attackers can obtain the circuit structure by reverse engineering. Then they may eliminate the effects of obfuscation logic and get the true CRPs of embedded PUF.

(3) Fake or extra CRPs to deceive the ML algorithm. Unlike the two secure-enhanced design methods above, this method focuses on poisoning and expanding the training dataset that attackers can obtain. There are two common ways to generate and broadcast fake CRPs. The first way is PUFs with dynamically changeable structures. Representative designs are adversarial attack-based APUF [21], NoPUF [22], CT PUF [23], dynamically configurable XOR APUF [24], etc. The second way is protocol-level deception using one or more fake PUF instances [25]. The CRPs data that the attacker obtains comes from several PUFs. So, it is hard to build a model of genuine PUF. This design method can be realized with simple PUF designs like the APUF and it does not decrease the performance of the PUF. So, it is interesting and broadly followed. But the ML and DL algorithm and computing power are developing very fast, which has empowered the robustness and noise-tolerant capacity of modeling algorithms.

As it stands, there are pros and cons to each of the three categories. We propose a dynamically configurable LFSR-driven obfuscation unit and a protocol-level two PUF instances mechanism to overcome these shortcomings.

3. DLFSR-based obfuscation APUF design

The proposed design structure is shown in Fig. 3. The original challenge $\boldsymbol{c}$ is loaded to DLFSR obfuscation units to generate the obfuscated challenge set $\langle \boldsymbol{c} \rangle_1$ which is then loaded to the n-stage APUF. The response of APUF is obfuscated to generate the final response $\boldsymbol{r}$. The obfuscation unit provides security against modeling attacks for the APUF, while the APUF provides unclonability.

3.1. DLFSR-based obfuscation unit

LFSR is a simple, fast, and easy-to-implement logical component. An n-bit length LFSR with primitive feedback polynomial structure can output a pseudo-random number sequence with a maximum period of 2^{n-1}. LFSR can be implemented with simple logic, so it has a small overhead in hardware. Some kinds of lightweight secure-enhanced strong PUF designs use LFSR to obfuscate the challenges and responses [3,26].

However, the complete output sequence of an n-bit LFSR can be computed as long as the successive $2n$-bit output is known using the Berlekamp–Massey algorithm [27]. Due to its intrinsic linearity, various cryptographic algorithms and strong PUF designs based on LFSR are not

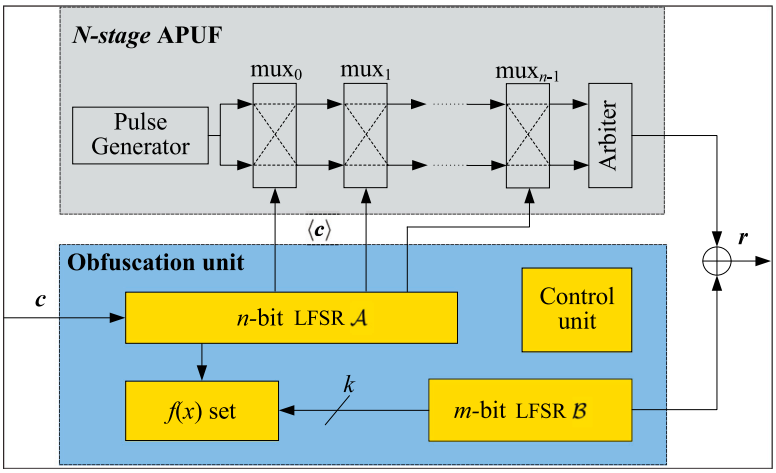

Fig. 3. The structure of the proposed strong PUF design.

cryptographically secure enough and are susceptible to fast algebraic and correlation attacks. One way to enhance the security of LFSR is to dynamically configure the feedback polynomial of LFSR.

The proposed obfuscation unit is constructed by two LFSRs: an n-bit length LFSR $\mathcal{A}$ with a cycle-changed feedback polynomial defined as $f(x)$. The next status of LFSR $\mathcal{A}$ is chosen from the pre-defined primitive polynomial set F randomly. An m-bit length LFSR $\mathcal{B}$ uses a standard clock drive and a fixed feedback polynomial defined as $g(x)$. In this mechanism, $g(x)$ must be a primitive polynomial to output the longest period. To obtain different outputs, the starting seed of LFSR $\mathcal{B}$ is different from each other. The original seed is written in the device using a one-time programmable (OTP) interface. k-bit from LFSR $\mathcal{B}$ drives the change of $f(x)$. In each authentication process, after $\boldsymbol{c}$ is loaded, the response is generated after n clock cycles running which is called the **initialization stage**. The purpose of this stage is to prevent the output of the LFSR from partially overlapping with $\boldsymbol{c}$, so that an attacker may get part of the real challenge. The obfuscation unit structure is shown in Fig. 4. The working mechanism of the proposed design after an n-bit challenge $\boldsymbol{c}$ passed to the device is described as follows:

- `Step1`: LFSR $\mathcal{A}$ runs for n clock cycles to initialize.
- `Step2`: LFSR $\mathcal{B}$ with feedback polynomial $g(x)$ runs for 1 clock cycle.
- `Step3`: The k bits from the output of LFSR $\mathcal{B}$ are sent into the decoder to have a 2^k-bit signal.
- `Step4`: The decoded signal is used to choose a primitive feedback polynomial $f(x)$ from F.
- `Step5`: The challenge $\boldsymbol{c}$ is sent into LFSR $\mathcal{A}$ with feedback polynomial $f(x)$.
- `Step6`: LFSR $\mathcal{A}$ runs for n clock cycles to generate a sub-challenge set $\langle \boldsymbol{c} \rangle$.
- `Step7`: $\langle \boldsymbol{c} \rangle$ is sent into the n-bit APUF instance to generate an n-bit output.
- `Step8`: The output of the APUF does a bitwise XOR operation with the output of $\mathcal{B}$ to get an n-bit final response $\boldsymbol{r}$.

For n-bit Galois type LFSR, the transition of register state $\mathbf{s} = (s_0, s_1, \ldots, s_{n-1})$ is determined by function T which is defined as:

$$\begin{pmatrix} s_0 \\ s_1 \\ \cdots \\ s_{n-2} \\ s_{n-1} \end{pmatrix} \rightarrow T \begin{pmatrix} s_0 \\ s_1 \\ \cdots \\ s_{n-2} \\ s_{n-1} \end{pmatrix} \rightarrow \begin{pmatrix} s_{n-1} \\ t(s_0) \\ \cdots \\ t(s_{n-3}) \\ t(s_{n-2}) \end{pmatrix}, \tag{2}$$

where t is a linear function on $GF(2)$.

$$t = f_i s_{n-1} \oplus s_i, i = 0, 1, \ldots, n-2, \tag{3}$$

f_i is the feedback polynomial of LFSR defined as:

$$f(x) = \sum_{i=0}^{n-1} f_i x^i, f_i \in (0, 1). \tag{4}$$

The feedback polynomial of LFSR $\mathcal{A}$ is not fixed. It is selected from primitive polynomial set F for each cycle using k bits of LFSR $\mathcal{B}$. The selecting scheme is determined by the structure and seed of LFSR $\mathcal{B}$, where F is:

$$F = \left\{ f_0(x), f_1(x), \ldots, f_{2^k-1}(x) \right\}.$$

For LFSR $\mathcal{A}$, the original register state is challenge $\boldsymbol{c}$. So, the transition situation of $\mathcal{A}$ in the initialization stage is as follows:

$$\begin{aligned} \boldsymbol{a}^{(0)} &= \boldsymbol{c} \\ \boldsymbol{a}^{(1)} &= T(\boldsymbol{c}) \\ \boldsymbol{a}^{(2)} &= T^2(\boldsymbol{c}) \\ &\cdots \\ \boldsymbol{a}^{(n)} &= T^n(\boldsymbol{c}) \end{aligned} . \tag{5}$$

After the initialization process, the value $\boldsymbol{a}^{n+1}$ is sent to the APUF instance then the n-bit response will be calculated after n cycles.

According to the attack model in Section 2, attackers are difficult to model the PUF because the true CRPs cannot be directly obtained. Furthermore, the chip's unique nonlinearity brought by a randomly chosen seed and configurable structure also prevents attackers model the PUF precisely.

3.2. Dual-PUF authentication protocol design

In the basic PUF-based authentication protocol, an attacker can impersonate the server and freely initiate a spoof authentication event, freely querying the PUF instance. She can model the PUF by repeatedly deploying a challenge or a set of selected challenges. This design uses a device-side nonce generator and a two-stage authentication mechanism based on the proposed PUF to avoid classical and reliability-based ML attacks. With two APUF instances, this protocol turns the next APUF circuit on only when the first authentication stage is successful. It makes true CRPs collecting more difficult. Design details are shown in Fig. 5. The specific steps are in the previous work [11].

(1) *Enrollment stage*. The enrollment stage is performed in a trusted environment before the devices are shipped out. Instead of the original CRPs set in the basic protocol, this protocol only stores pre-trained APUF models in the secure server. Then the OTP interface is permanently closed using irreversible fuses or tamper-proof public memory.

(2) *Authentication stage I*. This stage performs the device-to-server authentication process. When an authentication message from the

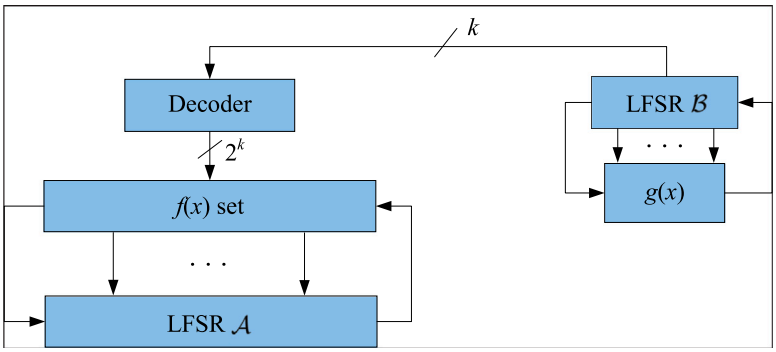

Fig. 4. The Structure of the obfuscation unit.

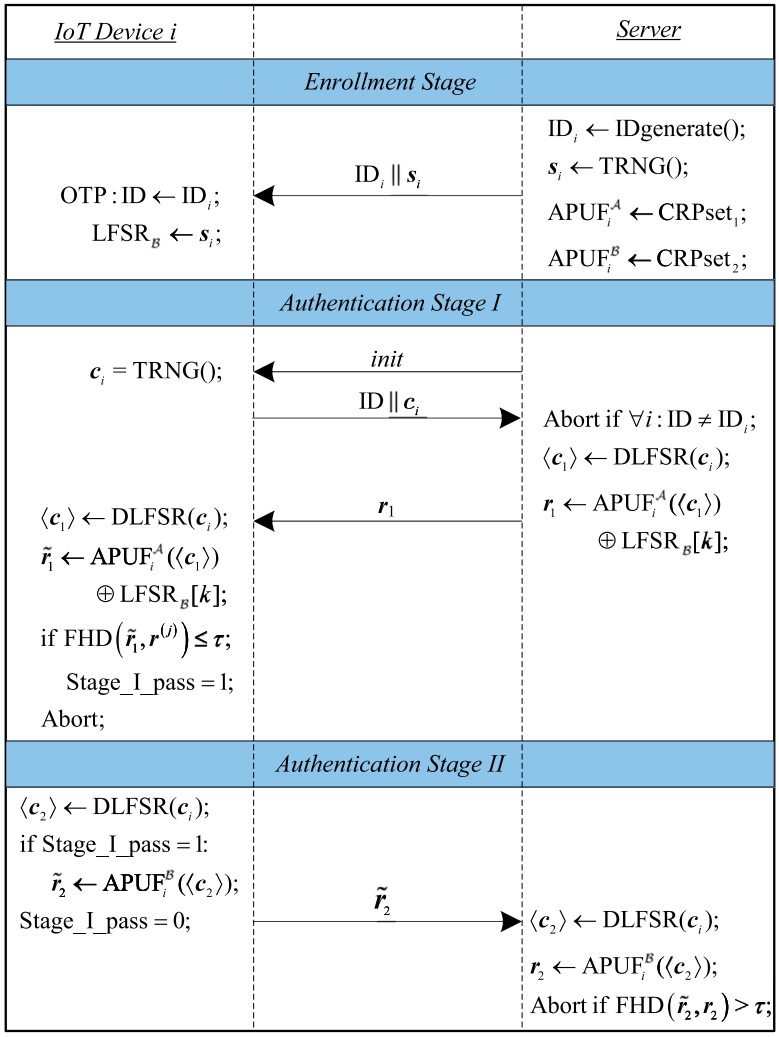

Fig. 5. The proposed protocol.

server is received by a device, it outputs a random nonce by an embedded true random number generator (TRNG). The nonce is sent to the authentication server and is operated with the enrollment data of this device to obtain a response r_1. The response r_1 is then sent back to the device. The device can authenticate the server by comparing r_1 with $\tilde{r}_1$.

(3) *Authentication stage II*. This stage performs the server-to-device authentication process. If stage I is successfully passed, the device deploys another sub-challenge set to the second APUF instance to generate $\tilde{r}_2$. The server can authenticate this device by comparing self-generating r_2 with $\tilde{r}_2$.

4. Security analysis

In this section, we further analyze and discuss the security of the proposed PUF and protocol. We first give an overview of the adversary model and discuss the recommended value of some security-related parameters. Then the resistance to modeling and protocol-level attacks is analyzed.

4.1. Adversary model

For an attacker Eve, the ultimate goal is to gain access to the secret information stored in the central server and take control of the network. For achieving this goal, she must try to pass authentication. Eve may impersonate the server or node appliance and try to deceive the opposite entity. For compromising the strong PUF protocols, Eve needs to know the circuit details of the inner PUF instances and construct numerical models of them within a polynomial period. If the model is well-trained, Eve can predict unseen CRPs and compromise the protocol. The attacking skills that Eve may have are as follows.

- *Query*($\mathbf{S}, \mathbf{m}, \boldsymbol{r}$): Eve tries to hack into the IoT network and impersonate a real appliance. Then she sends a query message $\mathbf{m}$ to the authentication server $\mathbf{S}$ and successfully receives a response $\boldsymbol{r}$.
- *Send*($\mathbf{D}, \mathbf{m}, \boldsymbol{r}$): Eve tries to hack into the IoT network and impersonate the authentication server $\mathbf{S}$. Then she sends message $\mathbf{m}$ to appliance $\mathbf{D}$, and stores the response $\boldsymbol{r}$ from $\mathbf{D}$.
- *Eavesdrop*($\mathbf{S}, \mathbf{D}$): Eve tries to hack into the IoT network and instantly eavesdrops on the communication channel between the authentication server $\mathbf{S}$ and a target appliance $\mathbf{D}$.
- *Block*($\mathbf{A}$): Eve tries to block the communication channel to break the information interaction on the channel.
- *Reveal*($\mathbf{D}$): Eve can obtain information in the NVM on the node appliance, e.g., by fault injection technology.
- *Reverse*($\mathbf{D}$): Eve can derive the internal circuit logic of a node appliance by reverse engineering technology. This attack is physically invasive and may destroy the device.

Based on their capabilities, potential attackers can be divided into two categories: Type I attackers can eavesdrop on the channel between the authentication server and target appliances. So, this type of attacker can freely use *Query*, *Send*, *Eavesdrop*, and *Block*. Type II attackers can manipulate node appliances. This type of attacker can perform physically invasive attacks on node appliances. So, this type of attacker can use all the attacking skills above. It is important to note that these skills have different overheads and costs. Communication channel attacks are commonly much less expensive than physical attacks. According to application scenarios and possible attackers, security protocol designers face a trade-off between security and overhead.

4.2. Parameters and security

For this configurable DLFSR-based PUF design, several configure parameters can affect the security performance. So, they should be well-configured depending on the application scenario.

(1) The length of LFSR $\mathcal{A}$

The goal of LFSR $\mathcal{A}$ is to obfuscate the original challenge. The length of LFSR $\mathcal{A}$ n directly determines the total number d of authentications as $n = \log_2 d$. This parameter primarily determines the ability of the protocol to resist replay attacks. Attackers will do *Query* and *Send* repeatedly until a used challenge appears. The possibility that a randomly generated challenge overlaps with a used one is 2^{-n}. The probability of success replay attack is $\Pr_{replay} = 2^{-n} \cdot (d-1)$. Normally, $n \geq 32$ is sufficient to avoid replay attacks.

(2) The length of LFSR $\mathcal{B}$ and chosen signal The most serious security issue of LFSR is its low linear complexity. If an attacker has access to a certain number of consecutive output bits, she can construct a whole pseudo-random sequence generated by the LFSR using a cryptanalysis algorithm. When the feedback polynomial of LFSR $\mathcal{B}$ is primitive, the output sequence period length is $s = 2^m - 1$. Since two n-bit responses $\boldsymbol{r}_1$ and $\boldsymbol{r}_2$ are required for each authentication, LFSR $\mathcal{A}$ needed to generate a $2n$ sub-challenge set. So, $s \geq 2n$ is required to avoid the sub-challenge overlap.

The k bits in the sequence are used to select a feedback polynomial for LFSR $\mathcal{A}$. The set of primitive polynomials that LFSR $\mathcal{B}$ can select is 2^k at most. The value of the k-bit transits randomly in an s-cycle period, and the probability of an attacker correctly guessing the value of the k-bit is $\Pr_{guess} = 2^{-k}$.

(3) Fault-tolerant threshold τ

The CRP of a PUF instance might be unreliable. Some bits of the response are affected by environmental factors like temperature and voltage fluctuations. Allowing an error bits threshold τ in the response is an effective way to reduce the fault rejection rate. Assuming the fault probability density of one response bit is $\Pr_e$ and all response bits follow a binomial distribution. The flip probability of τ-bit in n-bit response is:

$$\Pr_{noise} = \sum_{k}^{\tau \cdot n} \left(\begin{array}{c} n \\ k \end{array} \right) \cdot \left(\Pr_e\right)^k \cdot \left(1 - \Pr_e\right)^{n-k}. \tag{6}$$

The false rejection rate caused by noise is $\Pr_{fail} = 1 - \Pr_{noise}$. So, the value of τ should be set according to the requirements of PUF reliability and the protocol false rejection rate.

4.3. Security of obfuscated-APUF

In this part, the relation between the challenge and response of the obfuscation unit and APUF will be quantitatively analyzed.

Consider an obfuscated-APUF instance $\mathcal{P}$ with $n = 4, m = 4, k = 1$. The initial state of LFSR $\mathcal{A}$ is $\boldsymbol{c} = (c_0, c_1, c_2, c_3)$. The original seed of LFSR $\mathcal{B}$ is $\boldsymbol{o} = (o_0, o_1, o_2, o_3)$. The primitive polynomial set of LFSR $\mathcal{A}$ is: $F \in \{f_0(x) = x^4 + x + 1, f_1(x) = x^4 + x^3 + 1\}$, and the primitive polynomial of LFSR $\mathcal{B}$ is $g(x)$. Assuming b_3 is used to select feedback polynomial for LFSR $\mathcal{A}$ as $F = b_3 f_0(x) + (1 - b_3) f_1(x)$, b_1 is used to obfuscate the origin response of APUF. The state transition of LFSR $\mathcal{A}$ can be denoted as:

$$\begin{aligned} \mathbf{a}^{(1)} &= \left(c_3, t\left(c_0\right), t\left(c_1\right), t\left(c_2\right)\right) \\ \mathbf{a}^{(2)} &= \left(t\left(c_2\right), t\left(c_3\right), t^2\left(c_0\right), t^2\left(c_1\right)\right) \\ \mathbf{a}^{(3)} &= \left(t^2\left(c_1\right), t^2\left(c_2\right), t^2\left(c_3\right), t^3\left(c_0\right)\right) \\ \mathbf{a}^{(4)} &= \left(t^3\left(c_0\right), t^3\left(c_1\right), t^3\left(c_2\right), t^3\left(c_3\right)\right) \end{aligned}. \tag{7}$$

The state transition of LFSR $\mathcal{B}$ can be denoted as:

$$\begin{aligned} \mathbf{b}^{(1)} &= \left(o_3, t\left(o_0\right), t\left(o_1\right), t\left(o_2\right)\right) \\ \mathbf{b}^{(2)} &= \left(t\left(o_2\right), t\left(o_3\right), t^2\left(o_0\right), t^2\left(o_1\right)\right) \\ \mathbf{b}^{(3)} &= \left(t^2\left(o_1\right), t^2\left(o_2\right), t^2\left(o_3\right), t^3\left(o_0\right)\right) \\ \mathbf{b}^{(4)} &= \left(t^3\left(o_0\right), t^3\left(o_1\right), t^3\left(o_2\right), t^3\left(o_3\right)\right) \end{aligned}. \tag{8}$$

The feedback polynomial of LFSR $\mathcal{B}$ is denoted as $g(x)$. From (8) the selecting bit b_3 and the response obfuscating bit b_1 can be got:

$$\begin{aligned} b_3^{(1)} &= t\left(o_2\right) = g_2 o_3 \oplus o_2 \\ b_3^{(2)} &= t^2\left(o_1\right) = g_2\left(t\left(o_2\right)\right) \oplus t\left(o_1\right) \\ b_3^{(3)} &= t^3\left(o_0\right) = g_2\left(t^2\left(o_1\right)\right) \oplus t^2\left(o_0\right) \\ b_3^{(4)} &= t^3\left(o_3\right) = g_2\left(t^3\left(o_0\right)\right) \oplus t^2\left(o_3\right) \end{aligned}, \tag{9}$$

$$\begin{aligned} b_1^{(1)} &= t\left(o_0\right) = g_0 o_3 \oplus o_0 \\ b_1^{(2)} &= t\left(o_3\right) = g_0\left(t\left(o_2\right)\right) \oplus o_3 \\ b_1^{(3)} &= t^2\left(o_2\right) = g_0\left(t^2\left(o_1\right)\right) \oplus t\left(o_2\right) \\ b_1^{(4)} &= t^3\left(o_1\right) = g_0\left(t^3\left(o_3\right)\right) \oplus t^2\left(o_1\right) \end{aligned}. \tag{10}$$

From (7) and (9) we can get the first bit of $\mathbf{a}^{(4)}$ is:

$$\begin{cases} a_0^{(4)} = f_2^{(3)}\left(f_1^{(2)}\left(f_0^{(1)} c_0 \oplus c_3\right) \oplus t\left(c_2\right)\right) \oplus t^2\left(c_1\right) \\ f^{(i)}(x) = b_3^{(i)} f_0(x) + \left(1 - b_3^{(i)}\right) f_1(x), i = 1, 2, 3 \end{cases} \tag{11}$$

The value of $\mathbf{a}^{(4)}$ will be input to the APUF instance as the first challenge after the initialization stage. According to the delay model in (1), the first original response of APUF is:

$$r_{APUF} = \operatorname{sgn}\left(w \cdot \varphi\left(\mathbf{a}^{(i)}\right)\right), i \geq 4. \tag{12}$$

So, according to (10) and (12), the final response of the obfuscated-APUF is $r_{out} = r_{APUF} \oplus b_1^{(i)}, i \geq 4$.

Compared with the original APUF output model in (1), the relationship between challenge $\boldsymbol{c}$ and the response r_{out} of the obfuscated-APUF changes to a complex non-linear relationship. Moreover, the number of unknown coefficients and the order of the equation grow rapidly with n, m, and k. The attacker cannot obtain the genuine CRPs of the APUF. Although she can call a $Send(\mathbf{D}, \mathbf{m}, \boldsymbol{r})$ repeatedly to get a CRP set of the obfuscated-APUF unit and try to train a numerical model, the hyperplane of the sample space is far beyond the fitting capability of typical ML algorithms. The resistance to the modeling attack will be evaluated in Section 6.

4.4. Security of protocol

Even though the obfuscated-APUF is complex enough, the attacker still has a chance to compromise it if she can freely query the device and collect enough information. The proposed protocol can avoid the attacks on this aspect as the following analysis.

(1) Modeling attack

Modeling attack is the biggest threat to the protocol. For a regular modeling attack described in Section 2, the attacker must collect a CRP training set. There are two ways to achieve this goal. The first is using $Eavesdrop(\mathbf{S}, \mathbf{D})$ to capture previous authentication information as much as possible. Another way is trying to compromise the device and using $Send(\mathbf{D}, \mathbf{m}, \boldsymbol{r})$ to query the device freely. Type I attackers have this ability. In the proposed protocol, if the $Send(\mathbf{D_i}, \mathbf{m}, \boldsymbol{r})$ is called repeatedly, a Type I attacker Eve can obtain a set of challenges $\boldsymbol{c}_i$. She uses these challenges to try to receive related responses from the server by $Query(\mathbf{S}, \mathbf{m}, \boldsymbol{r})$. The server is considered secure in the adversary model and will forbid illegal continuous queries to the server by server-side protection mechanisms. For an APUF, she may only need to use $Eavesdrop(\mathbf{S}, \mathbf{D})$ 10 times to get enough training data. But for an obfuscated-APUF, it is impossible this way. So, authentication stage I can resist attackers to derive the model of APUF $\mathcal{A}$. Even if Eve uses $Eavesdrop(\mathbf{S}, \mathbf{D})$ operation and obtains some previously used $\boldsymbol{r_1}$. Although she may continuously deploy *init* and $\boldsymbol{r_1}$ on the appliance to try to open the Stage_I_pass controller, the TRNG can avoid it, and she cannot contact APUF $\mathcal{B}$.

Recently, a reliability-based modeling method is proposed and has a significant result in attacking XOR APUF [19]. The main idea is that the reliability of a response bit can imply the inherent information of an APUF. A small delay difference may generate an unstable response, while a bigger time delay difference may generate a relatively stable response. By deploying different challenges multiple times, the attacker can evaluate error frequencies and then uses an ML algorithm to derive a model of APUF. In the proposed protocol, both the server and the device ends have mechanisms to prevent the attacker's repeated access. On the server end, the built-in security mechanism can ensure that the same group challenges cannot be reused. On the device end, the dynamic dual LFSR mechanism can ensure that one same group challenges cannot generate the same responses when repeatedly inputted. Although Eve can use a fixed challenge $\boldsymbol{c}$ several times, she cannot get information from both the server and devices. This reliability-based attacking method is useless.

(2) Replay attack

An attack method called replay attack may be performed by attackers who obtain real used authentication data. In the proposed protocol, the authentication server marks a CRP when it is used and does a repeatability check when a new authentication begins. This mechanism ensures that a CRP is never used twice.

(3) Spoofing attack

As mentioned before, attackers may impersonate the authentication server or target device to perform deception. The proposed protocol implements mutual authentication by using a well-trained APUF model stored on the server. It ensures that all entities are trusted only by the two-stage authentication passing. So, the spoofing attack is effectively denied.

5. Physical security enhancement

Many obfuscation-based PUF or protocol designs are vulnerable to physical attack. Some of them need inside NVM to store some secret information. Attackers can use $Reveal(\mathbf{D})$ to steal information from NVM. Some of them use fix-structure obfuscation logic. The logic details are exposed if attackers can use $Reverse(\mathbf{D})$. Physical attacks are so expensive and destructive that they are not used on cheap IoT devices. But for crucial applications like military sensors, adversaries will hack the system regardless of the cost, and physical security becomes more important. So, we propose a possible physical attack model and a variant protocol. The variant protocol can effectively resist physical attacks.

5.1. Attack model of physical attack

Considering an extreme scenario, a Type II attacker Eve can use $Reveal(\mathbf{D})$ and $Reverse(\mathbf{D})$ at will. She can impersonate the node device by modeling the obfuscated-APUF as follows:

- `Step1`: Eve uses $Eavesdrop(\mathbf{S}, \mathbf{D_i})$ continuously to collect d groups of $(\boldsymbol{c}, \boldsymbol{r_1}, \tilde{\boldsymbol{r}}_2)$.
- `Step2`: Eve uses $Reveal(\mathbf{D_i})$ model to get the seed of LFSR $\mathcal{B}$.
- `Step3`: Eve uses $Reverse(\mathbf{D_i})$ model to obtain all circuit details including feedback polynomials of two LFSRs. The device i is destructed.
- `Step4`: Using the data collected in `Step1` and the circuit details, Eve can compute and get the original CRPs of APUF. Then she can train a precise model of two APUFs.
- `Step5`: Now Eve can impersonate device i in the network and respond to the server's authentication request.

The protocol proposed in Section 3 is called Protocol I. If an adversary can do attacks as above, she can easily compromise Protocol I and spoof the server to hack into the network.

5.2. Physical attack resist protocol

To fix this problem, an improved Protocol II which can resist the attack method described above is proposed. As shown in Fig. 6, only m bits in LFSR $\mathcal{B}$ initial seed come from the enrollment stage. A random pattern is generated with l bits from the device-side TRNG. The steps of Protocol II are shown in Fig. 7.

(1) *Enrollment stage*. The enrollment stage is almost the same as Protocol I. The only difference is `Step2`. Only $(m - l)$ bits are needed to generate the seed of LFSR $\mathcal{B}$ for each device.

(2) *Authentication stage I*. When the *init* signal has arrived, device i will generate an n-bit challenge $\boldsymbol{c}_i$ and an l-bit sub-seed for LFSR $\mathcal{B}$. When the server receives $\boldsymbol{c}_i$, it calculates $j = 2^l$ groups of sub-challenges. Then the server deploys all sub-challenges to $\widehat{APUF_i^{\mathcal{A}}}$ to generate j responses. All the responses are randomly shuffled and sent back to device i. The device will do a comparison up to j times until the right response is found. Otherwise, the process will be terminated.

(3) *Authentication stage II*. When stage I is completed, the device i will send the response of $APUF_i^{\mathcal{B}}$ to the server. The server will do a comparison up to j times until the right response is found. Otherwise, the authentication process will be aborted.

5.3. Security analysis

According to the attack model, Eve can eavesdrop on the communication between the server and device i to collect d groups of $\boldsymbol{c}, \mathbf{r}_1^{(1)} \| \ldots \| \mathbf{r}_1^{(j)}, \tilde{\boldsymbol{r}}_2$. Then she compromises the device i and tries to train models of APUFs.

Although Eve has known details of the device and authentication data, she does not know the l-bit initial seed of LFSR $\mathcal{B}$. She can use

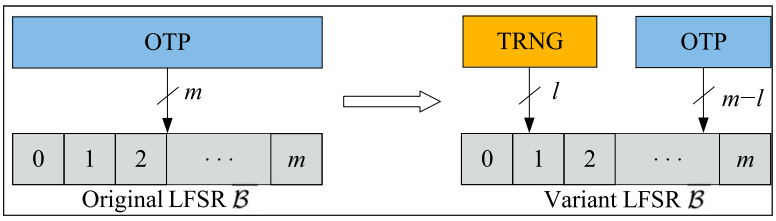

Fig. 6. A random pattern variant of LFSR $\mathcal{B}$.

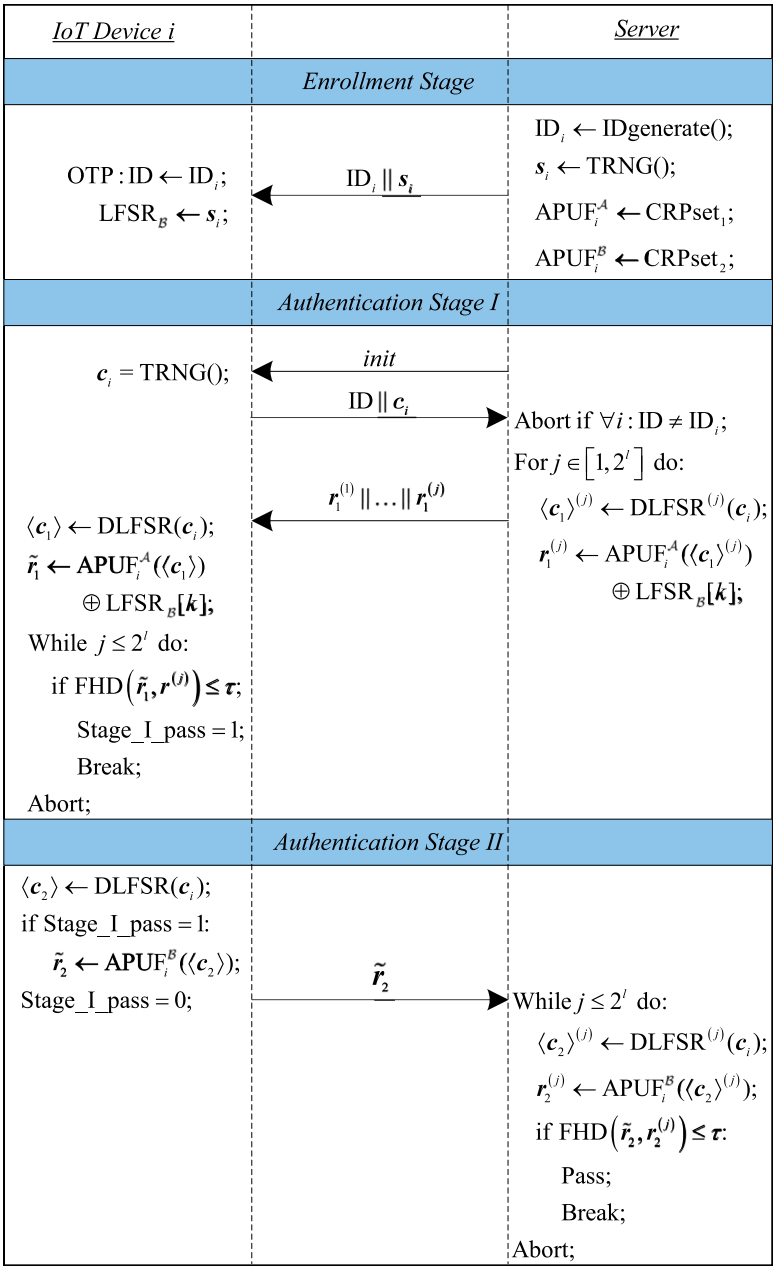

Fig. 7. A variant protocol II.

c_i to generate 2^l kinds of sub-challenge sets, but she does not know which is the right one. Of course, she can do random guesses. The probability of successfully guessing is $\Pr_{crp\_guess} = 2^{-l}$. Assuming all the d CRPs are following independently identically distribution, the probability of successfully guessing all d CRPs is $\Pr_{crps\_guess} = 2^{-ld}$. As mentioned in Section 1, Eve needs 640 CRPs to model a 64-bit APUF. She needs $d = 10$ times authentication data at least. If $l = 2$, the probability $\Pr_{crps\_guess} = 2^{-20}$, which is equivalent to about 3 h to break the system if it takes 0.01 s to train a model. If $l = 4$, this period is about 350 years. Of course, ML algorithms can tolerate some noise in the training set. Wrong guesses can be considered noise or dirty training data. The proportion of noise in the data is $1 - 1/2^l$. If $l = 1$, the noise

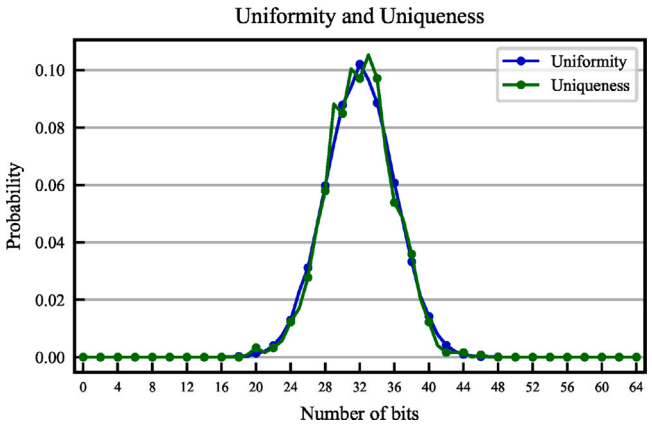

Fig. 8. HW and HD_{inter} distribution of 50 PUF instances.

proportion is 1/2. It greatly increases the variance of ML algorithms and causes underfitting.

Protocol II can effectively resist the physical attack with some extra hardware and communication resources. In most IoT application scenarios, Protocol I is secure enough. When physical security is necessary, Protocol II is a better choice.

6. Experimental results and analysis

To evaluate the performance of the proposed obfuscation unit and authentication protocol, simulations were performed in software and hardware, respectively. The obfuscated-APUF was simulated in Python, similar to the approach adopted in [19,28]. Then, two protocols were implemented on FPGA.

6.1. Performance of obfuscated-PUF

Considering that all delay weights are independent and identically distributed. They follow a normal distribution with $\mu = 10$ and $\sigma = 0.05$. 50 PUF instances with $n = 64$, $m = 8$ and $k = 2$ are simulated in Python. To evaluate the reliability metric of these PUF instances, different scale noises are added to weight vectors. Noises follow a normal distribution with $\mu = 10$ and $\sigma_{noise} = \alpha\sigma$, where α is the noise level. Each PUF instance is measured 25 times with 1×10^4 challenges under 3 different noise levels. For comparison, 64-bit APUF is simulated and evaluated under the same conditions.

(1) *Uniformity*. The uniformity of response represents the randomness of PUF. It can be estimated by the fraction of 1's in response. The ideal value of uniformity is 50%. The value of uniformity can be calculated by Hamming weight (HW) of responses. The HW distribution of obfuscated-APUF is shown in Fig. 8. The average (Avg.) and standard deviation (Std.) values are listed in Table 1.

(2) *Uniqueness*. The uniqueness of the PUF indicates the distinction between different PUF instances. It can be considered a quantitative representation of unclonability. The ideal value of uniqueness is 50%. The uniqueness can be calculated by inter-chip Hamming distance (HD_{inter}) as:

$$Uniqueness = \frac{2}{d\,(d-1)} \sum_{i=1}^{d-1} \sum_{j=i+1}^{d} \frac{HD\left(\boldsymbol{r}^i, \boldsymbol{r}^j\right)}{n} \times 100\%, \tag{13}$$

where d is the number of PUF instances, $\boldsymbol{r}^i$ and $\boldsymbol{r}^j$ are response vectors of instances i and j with the same challenge.

The HD_{inter} distribution is shown in Fig. 8. The average (Avg.) and standard deviation (Std.) values are listed in Table 1.

(3) *Reliability*. To design a truly practical PUF circuit, acceptable reliability performance is necessary. Ideally, the output response of a PUF should be completely reproducible. However, due to temperature and supply voltage variations in the operating environment, it is difficult to achieve ideal reliability. The reliability can be estimated by the intra-chip Hamming distance (HD_{intra}). HD_{intra} is the average HD between responses under different environmental conditions which are generated by the same PUF instance with a fixed challenge. The HD_{intra} of PUF instance i is:

$$HD_{\text{intra}} = \frac{1}{s} \sum_{t=1}^{s} \frac{HD\left(r^i, r_t^{i\prime}\right)}{n}, \tag{14}$$

where s is the number of measurements of the same challenge $\boldsymbol{c}$ at different noise levels α.

There are three main ways to improve reliability: highly reliable PUF designs, error-correcting circuits, and application-layer fault tolerance. Due to the intrinsic of PUFs, security-improving technologies usually amplify unreliability. So, the correction algorithms are necessary, and the hardware overhead is increased with it. However, the obfuscation logic methods have less effect on the reliability of PUFs. If a highly reliable APUF circuit is used as in [29], the overhead of the whole protocol will be acceptable with fault-tolerant mechanisms.

The obfuscation unit proposed in this paper has a fixed function and is unaffected by the operating environment. So, the reliability of the proposed obfuscated-APUF has the same reliability as the APUF inside. It is an advantage that enhances the security of PUF without reliability loss. The reliability under 3 different noise levels is listed in Table 1.

6.2. Resilience to modeling attacks

Recently, many researches tried to deploy modeling attacks on popular strong PUF designs. Some of them have impressive results. These works can mainly be categorized into three classes: ML-based attack, DL-based attack, and evolution strategy (ES) attack. The resistance to modeling attacks of the proposed design was evaluated using all three attacking methods. The software environment is Python 3.9 and PyTorch 1.10.0. The hardware platform is 3.5 GHz Intel Core i9-11900K with 64 GB RAM and NVIDIA GeForce RTX 3090. We collected 1×10^5 real CRPs as a sample dataset including the training and testing sets. At the end of the section, some discussions about the attacking methods are conducted.

(1) *ML-based attack resistance*. Logistic Regression (LR) and Support Vector Machine (SVM) are well-known supervised ML algorithms. They can be used as linear classifiers to approximate the model of PUFs. For the LR algorithm, a sigmoid function has been used as the classification function, and several linear and non-linear polynomials are used to decrease the underfitting result. Then the RProp gradient descent is used as the optimization method. For the SVM algorithm, the RBF kernel function is used to figure nonlinearity in this design. The binary cross entropy between the predicted value and real value is used as the loss function in both two attacking algorithms. The learning results are shown in Table 2. The training set has been cross trained and validated for 1000 epochs. It shows that the prediction accuracy rates are up to 99.86% and 94.98% only using 1×10^4 CRPs, but the proposed design has no convergence on the full dataset up to the max iteration.

(2) *DL-based attack resistance*. In recent years, with the rapid development of artificial intelligence technology, DL techniques have proven their outstanding advantages for complex learning tasks, especially in the fields of computer vision and natural language processing. The feed-forward artificial neural network (ANN) is one of the most popular DL algorithms. The hyperparameters of the attacking ANN are listed in Table 3 and the learning results are shown in Table 4.

(3) *ES attack resistance*. This method is a heuristic algorithm based on population which can do attacking silicon PUF with known logic function. The training process begins with a series of parameters and real CRP samples of the PUF instance. By using specific evolution strategies and well-chosen fitness functions, it could find the best

Table 1
The uniformity, uniqueness, and reliability of the 64-bit APUF and the Obfuscated-APUF.

PUF	Challenge	Noise α	Uniformity (Avg., Std.)	Uniqueness (Avg., Std.)	Reliability (Avg., Std.)
APUF	64-bit	1/2	(47.91,4.09)	(50.02,4.03)	(82.79, 2.94)
		1/20	(49.83,4.12)	(50.07,3.99)	(98.25,1.02)
		1/80	(49.07,4.16)	(49.54,4.02)	(99.51,0.57)
Obfuscated-APUF	64-bit	1/2	(49.91,3.99)	(50.03,3.98)	(85.33,3.05)
		1/20	(49.95,4.04)	(50.12,4.02)	(98.40,1.01)
		1/80	(50.07,4.00)	(50.14,4.04)	(99.60,0.51)

Table 2
Modeling attack results of rwo ML algorithms.

PUF	Challenge	CRPs	LR predict rate	SVM predict rate
APUF	64-bit	10000	99.86%	94.98%
Obfuscated-APUF	64-bit	100000	50.53%	50.59%

Table 3
Hyperparameters of ANN.

Hyperparameters	Value
Learning rate	0.01
Optimizer	Adam
Loss function	Binary cross-entropy
Hidden layers	4
The hidden layer activation function	ReLU
Output layer activation function	Sigmoid

Table 4
Modeling attack results of ANN.

PUF	Challenge	CRPs	Predict rate
APUF	64-bit	10000	94.38%
Obfuscated-APUF	64-bit	100000	50.33%

Table 5
Modeling attack results of the CMA-ES algorithm.

PUF	Challenge	CRPs	Generations	Predict rate
APUF	64-bit	10000	100	94.69%
Obfuscated-APUF	64-bit	100000	200	50.58%

parameters that can precisely fit the PUF models after some generations of evolution. The CMA-ES is a powerful ES algorithm that can efficiently compromise some APUF-based designs [30]. It has many open-source implementations and we use a lightweight implementation for Python 3 (https://pypi.org/project/cmaes/). The dimension size of one population in the ES algorithm is chosen according to the model of PUFs. Responses are generated using real challenges and attributes of the populations per generation. The fitness of populations is evaluated by the Pearson correlation coefficient between real responses and generated responses. APUF has a simple structure and precise function model. The ES attack on 64-bit APUF can start from a 256-dimension ancestor (each stage in the delay path has 4 delay parameters). For the proposed obfuscated-APUF, the numeral model is more complex. A 336-dimension ancestor is initially generated (256 parameters for the 64-bit APUF, 64 parameters for taps of LFSR $\mathcal{A}$, and 16 parameters for taps and the seed of LFSR $\mathcal{B}$). The learning results are shown in Table 5. It shows that the APUF prediction accuracy is up to 94.69% in 100 generations using 1×10^4 CRPs, but the proposed design is not vulnerable to the ES attack method.

(4) *Some discussions*. It needs to note that the ML-based attacks are conducted in black-box-like ways. Currently, only the mathematical model of APUF is simple and accurate, while the structures of most heterogeneous PUFs are different, and some designs lack mathematical tools to support the principles they employ. Therefore, it is relatively complex to model them mathematically and select appropriate attack algorithms. In fact, relatively simple binary ML algorithms such as LR and SVM, are unlikely to have a good attack effect on most modern PUF designs because the problems they can solve are relatively simple. An important principle of PUF design is to increase non-linear features as much as possible without sacrificing reliability. Therefore, for basic ML algorithms, there are limited sample distribution modes that can be fitted. The mathematical model of the design in this paper has been derived previously. Compared with APUF and its basic variants, the complexity of this mathematical model is much higher. Using basic ML classification algorithms such as LR and SVM for attacks is actually a black-box attack, which cannot precisely fit its mathematical model.

For attacking any PUF design, DL algorithms should be more advantageous because the structure and quantity of neural networks can be freely adjusted and changed. According to the universal approximation theorem, a fully connected neural network with two or more layers can fit any function, not to mention new seq-to-seq models such as Transformer. Despite the large number of model and hyperparameter tuning experiments required, using DL for attack and defense is a very interesting area of research.

Approximation attacks are a customized deep learning attack method that compromised MPUF perfectly [31]. We have analyzed this new attack method. Because the core part dynamic configured LFSR has complex iterating sequential logic, it is difficult to decompose DLFSR-APUF into basic logical gates to apply logical approximation attack and deployed attack. But as mentioned above, a sophisticated-designed deep neural network model may have better efficiency on complex non-linear PUF modeling with more powerful hardware. This work should be further studied in the future.

6.3. Implementation and comparison

The proposed obfuscated-APUF and authentication protocols were implemented on an FPGA evaluation board Xilinx Zedboard Zynq-7000 xc7z020 with an embedded Artix-7 chip. The logic function of the device-side is depicted in Fig. 9. The APUF structure is hard to implement on FPGA because the routing strategy is impossible to generate fully symmetrical delay paths. Fortunately, a design method called programmable delay line (PDL) can be used to implement available APUF circuits [32]. So, the inner APUF instances are implemented by using PDL. For reducing hardware overhead, a timing jitter-based TRNG design called ES-TRNG is implemented as inner TRNG. The ES-TRNG only spends 10 LUTs and 5 FFs on the Xilinx Spartan-6 chip [33].

(1) *Hardware overhead*. The core circuit uses 127 of the 53 200 LUTs or 0.24% of total available LUT resources; 186 of the 106 400 FFs, or 0.17% of total available FF resources; 6 of the 17 400 memories are used to store chip ID. Specific resource usage is shown in Table 6.

The proposed design is compared with several recent designs in the same category. The resource usage of two classic cryptographic techniques, a fast version of 128-bit Advanced Encryption Standard (AES) and a secure Hash algorithm (SHA-256 [34]) on the same board are also listed. The results in Table 7 show that the proposed protocol is sufficiently lightweight for the same challenge length in terms of the overall usage of LUTs and FFs, and the PUF-based secure mechanism has a great advantage on hardware overhead.

The 64-bit Protocol I for general security requirements accounts for 0.52% of the total available resources and the 128-bit Protocol I with higher security accounts for 0.92%. The 64-bit Protocol II, which is

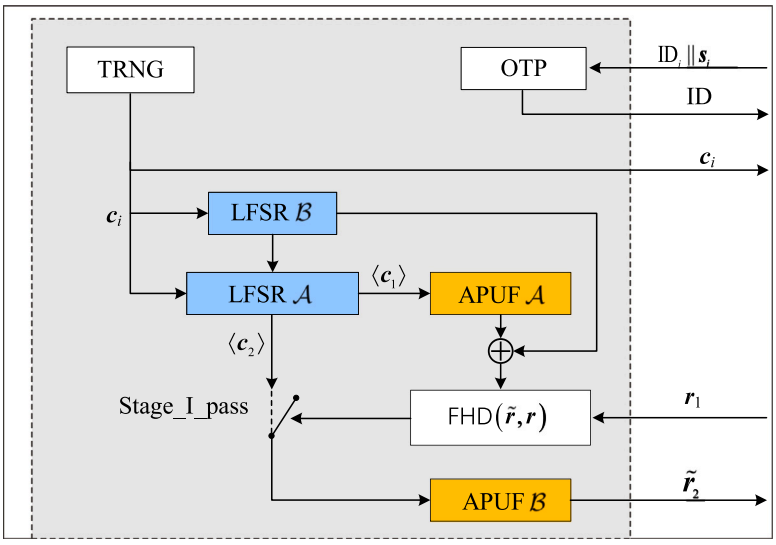

Fig. 9. The logic function of the device-side.

Table 6
Resource usage and power consumption of the device.

Protocol type	(m, n, k, l[a])	Function type	LUT	FF	Power (W)
Protocol I	(8, 64, 2)	PUF	108	75	0.204
		Control unit	52	154	
		Total	160(0.3%)	229(0.22%)	
	(8, 128, 2)	PUF	204	139	0.335
		Control unit	73	282	
		Total	277(0.52%)	421(0.4%)	
Protocol II	(8, 64, 2, 1)	PUF	108	75	0.331
		Control unit	175	294	
		Total	283(0.53%)	369(0.35%)	
	(8, 128, 2, 1)	PUF	204	139	0.336
		Control unit	305	550	
		Total	509(0.96%)	689(0.65%)	

[a]Note that only Protocol II has the parameter l.

Table 7
Comparison with same category designs and standard cryptographic primitives.

Designs	Challenge(bit)	LUT	FF	Total	Relative Ratio
SHA-256	~	3756	2609	6365	1636%
AES [35]	128	1642	820	2462	633%
The improved Protocol A[a] [36]	64	564	345	909	234%
Protocol B[a] [36]	64	2024	1316	3330	856%
Deception-based Protocol [25]	64	578	784	1362	350%
PUF-RAKE Protocol [37]	64	636	592	1228	316%
Protocol I	64	160	229	389	100%
Protocol I	128	277	421	698	179%
Protocol II	64	283	369	652	168%
Protocol II	128	509	689	1198	308%

[a]The protocol A and B in [36] are implemented on Xilinx Virtex-5.

resistant to physical attacks, accounts for 0.88% of the total available resources, and the 128-bit Protocol II, which has the highest security, accounts for 1.61%, or 308% of the 64-bit Protocol I overhead.

In a real application scenario, the configuration of parameters in two protocols needs to consider the tradeoff between hardware overhead, manufacturing cost, and security requirements. These parameters include the feedback polynomials $f(x)$ (selected from F) and $g(x)$ for two LFSRs, the (m, n, k, l) set, the embedded PUF structure, and so on. For instance, there are three possible ways to configure the two LFSRs as follows:

- All PUF instances have the same F and $g(x)$, and the initial seed for $g(x)$ is the same. This has the lowest cost but the poorest security. An attacker can get the internal structure of the PUF design by reverse-engineering one instance.
- All PUF instances have the same F and different initial seeds for $g(x)$. Different seeds are configured for each instance using OTP, so that each instance selects a different $f(x)$ for each authentication process, improving security. However, since LFSR $\mathcal{B}$ only has 8 bits, if the number of instances far exceeds $2^8 = 256$, there are still a significant number of instances with identical structures.
- All PUF instances have different F and $g(x)$. This has the highest cost and requires selecting different structures for each instance to achieve the highest level of security. However, this is practically infeasible in engineering. A better configuration method is to divide the instances into different batches, with each batch having the same F and $g(x)$ configuration, while the configuration differs between different batches.

Table 8
Comparison of communication cost.

Designs	Challenge (bit)	Communication cost (bit)
The improved Protocol A [36]	64	$128 \times k$
Protocol B [36]	64	$384 + 128 \times k$
Deception-based Protocol [25]	64	416
PUF-RAKE Protocol [37]	64	576
Ideal-PUF Protocol [42]	64	640
Noise-PUF Protocol [42]	64	1792
Protocol I	64	$128 + 64 \times 3 = 320$
Protocol II	64	$128 + 64 \times (j + 1)$

Furthermore, the (m, n, k, l) set configuration of the protocol can be chosen according to the application's needs. Some new and excellent lightweight PUF designs like PI PUF [38], DA PUF [39], MC-PUF [40], Shadow PUFs [41] can be used to replace the APUF in this design to further decrease the hardware overhead.

(2) *Communication overhead*. Due to the special deployment environment, IoT applications are usually sensitive to environmental variation and bandwidth. So, the communication overhead is an important performance metric for IoT device authentication protocols. It is computed by the amount of data transferred in one complete authentication process. Table 8 lists the communication overhead of several same-category protocols and the protocols proposed in this paper. They all have a 128-bit device ID and an embedded 64-bit PUF. The results indicate that the two proposed protocols are better performance in terms of communication overhead compared to other protocols.

Table 9
Security comparison under different secure metrics.

Protocols	SM1	SM2	SM3	SM4	SM5	SM6
The improved Protocol A [36]	no	yes	no	no	no	N
Protocol B [36]	yes	yes	no	no	yes	∞
Deception-based Protocol [25]	yes	yes	yes	no	no	∞
PUF-RAKE Protocol [37]	yes	yes	no	no	yes	∞
Ideal-PUF Protocol [42]	yes	yes	no	yes	yes	∞
Noise-PUF Protocol [42]	yes	yes	no	yes	yes	∞
Protocol I	yes	yes	no	yes	no	∞
Protocol II	yes	yes	yes	yes	no	∞

SM1: Mutual Authentication; SM2: Modeling Attack Robust;
SM3: Physical Attack Robust; SM4: DoS Robust;
SM5: Crypto Primitive; SM6: Number of Authentication

(3) *Protocol level security*. The protocol level security performance is evaluated by some main security metrics (SM) listed in Table 9. Protocol I can meet all the important SMs except for physical invasive attacks, and Protocol II can meet all the SMs.

7. Conclusion

In this article, we first proposed a lightweight obfuscation unit design using two LFSRs. It can build a secure strong PUF combined with an APUF. Subsequently, based on the obfuscated-APUF, we proposed a lightweight mutual authentication protocol that can resist modeling attacks. We analyzed the security of the obfuscated PUF and protocols. Then we improved the protocol and proposed a security-enhanced variant that can effectively resist physical attacks. Experimental analyses show that the proposed protocols ensure the desired security requirements efficiently by using the dynamic obfuscating logic and two APUF instances. Specifically, the protocol maintains less hardware overhead and communication costs. Overall, the performance of the proposed technologies is better than those of other existing PUF-based authentication protocols in IoT applications. Therefore, these protocols are more suitable for designing a secure IoT system.

CRediT authorship contribution statement

Shen Hou: Conceptualization, Methodology, Data analysis, Writing – original draft. **Yanzhou Ma:** Supervision, Funding acquisition, Writing – review & editing. **Ding Deng:** Methodology, Data curation, Formal analysis. **Zhenyu Wang:** Software, Data curation, Visualization. **Guolei Ren:** Supervision, Writing – review & editing.

Declaration of competing interest

All authors disclosed no relevant relationships.

Data availability

No data was used for the research described in the article.

Acknowledgment

This work is supported by the National Natural Science Foundation of China under Grant No 61832018.

References

[1] Number of IoT 2022, number of Internet of Things (IoT) connected devices worldwide 2022/2023: Breakdowns, growth & predictions. 2020, https://financesonline.com/number-of-internet-of-things-connected-devices/.
[2] Guajardo J, Kumar SS, Schrijen G-J, Tuyls P. In: Paillier P, Verbauwhede I, editors. Cryptographic hardware and embedded systems, vol. 4727. Berlin, Heidelberg: Springer Berlin Heidelberg; 2007, p. 63–80. http://dx.doi.org/10.1007/978-3-540-74735-2_5.
[3] Delvaux J, Peeters R, Gu D, Verbauwhede I. ACM Comput Surv 2015;48(2):1–42. http://dx.doi.org/10.1145/2818186.
[4] Ruhrmair U, van Dijk M. 2013 IEEE symposium on security and privacy. Berkeley, CA: IEEE; 2013, p. 286–300. http://dx.doi.org/10.1109/sp.2013.27.
[5] Ronen E, Shamir A, Weingarten A-O, OFlynn C. 2017 IEEE symposium on security and privacy. San Jose, CA, USA: IEEE; 2017, p. 195–212. http://dx.doi.org/10.1109/sp.2017.14.
[6] Konigsmark STC, Chen D, Wong MDF. IEEE Trans Comput-Aided Des Integr Circuits Syst 2016;35(7):1053–66. http://dx.doi.org/10.1109/tcad.2015.2488493.
[7] Gao Y, Li G, Ma H, Al-Sarawi SF, Kavehei O, Abbott D, et al. 2016 IEEE international conference on pervasive computing and communication workshops. 2016, p. 1–6. http://dx.doi.org/10.1109/PERCOMW.2016.7457162.
[8] Ye J, Hu Y, Li X. 2016 IEEE Asian hardware-oriented security and trust. 2016, p. 1–6. http://dx.doi.org/10.1109/asianhost.2016.7835567.
[9] Idriss T, Bayoumi M. 2017 IEEE international conference on RFID technology & application. Warsaw: IEEE; 2017, p. 214–9. http://dx.doi.org/10.1109/rfid-ta.2017.8098893.
[10] Gao Y, Ma H, Al-Sarawi SF, Abbott D, Ranasinghe DC. IEEE Trans Comput-Aided Des Integr Circuits Syst 2017;37(5):1104–8. http://dx.doi.org/10.1109/tcad.2017.2740297.
[11] Ren G, Hou S, Wang X. 2021 2nd international conference on electronics, communications and information technology. 2021, p. 71–5. http://dx.doi.org/10.1109/CECIT53797.2021.00020.
[12] Gassend B, Lim D, Clarke D, van Dijk M, Devadas S. Concurr Comput: Pract Exper 2004;16(11):1077–98. http://dx.doi.org/10.1002/cpe.805.
[13] Kumar R, Burleson W. 2014 IEEE 32nd international conference on computer design. Seoul, South Korea: IEEE; 2014, p. 493–6. http://dx.doi.org/10.1109/iccd.2014.6974725.
[14] He Z, Wan M, Deng J, Bai C, Dai K. IEEE Trans Very Large Scale Integ (VLSI) Syst 2018;26(6):1073–83. http://dx.doi.org/10.1109/tvlsi.2018.2806041.
[15] Afghah F, Cambou B, Abedini M, Zeadally S. Int J Wirel Inf Netw 2018;25(2):117–29. http://dx.doi.org/10.1007/s10776-018-0391-6.
[16] Machida T, Yamamoto D, Iwamoto M, Sakiyama K. 2014 Federated conference on computer science and information systems. 2014, p. 871–8. http://dx.doi.org/10.15439/2014f140.
[17] Sahoo DP, Mukhopadhyay D, Chakraborty RS, Nguyen PH. IEEE Trans Comput 2018;67(3):403–17. http://dx.doi.org/10.1109/tc.2017.2749226.
[18] Nguyen PH, Sahoo DP, Jin C, Mahmood K, Rührmair U, van Dijk M. IACR Trans Cryptogr Hardw Embed Syst 2019;243–90. http://dx.doi.org/10.13154/tches.v2019.i4.243-290.
[19] Becker GT. In: Güneysu T, Handschuh H, editors. Cryptographic hardware and embedded systems, vol. 9293. Berlin, Heidelberg: Springer Berlin Heidelberg; 2015, p. 535–55. http://dx.doi.org/10.1007/978-3-662-48324-4_27.
[20] Delvaux J. IEEE Trans Inf Forensics Secur 2019;14(8):2043–58. http://dx.doi.org/10.1109/tifs.2019.2891223.
[21] Wang S-J, Chen Y-S, Li KS-M. IEEE J Emerg Sel Top Circuits Syst 2021;11(2):306–18. http://dx.doi.org/10.1109/JETCAS.2021.3062413.
[22] Wang A, Tan W, Wen Y, Lao Y. IEEE Trans Circuits Syst I Regul Pap 2021;68(6):2508–21. http://dx.doi.org/10.1109/tcsi.2021.3067319.
[23] Zhang J, Shen C, Guo Z, Wu Q, Chang W. IEEE Internet Things J 2021;1. http://dx.doi.org/10.1109/jiot.2021.3090475.
[24] Wang Y, Wang C, Gu C, Cui Y, O'Neill M, Liu W. IEEE Trans Emerg Top Comput 2021;1. http://dx.doi.org/10.1109/tetc.2021.3072421.
[25] Gu C, Chang C-H, Liu W, Yu S, Wang Y, OrNeill M. IEEE Trans Comput-Aided Des Integr Circuits Syst 2020;1. http://dx.doi.org/10.1109/tcad.2020.3036807.
[26] Yu M-D, Hiller M, Delvaux J, Sowell R, Devadas S, Verbauwhede I. IEEE Trans Multi-Scale Comput Syst 2016;2(3):146–59. http://dx.doi.org/10.1109/tmscs.2016.2553027.
[27] Massey J. IEEE Trans Inform Theory 1969;15(1):122–7. http://dx.doi.org/10.1109/tit.1969.1054260.
[28] Ruhrmair U, Solter J, Sehnke F, Xu X, Mahmoud A, Stoyanova V, et al. IEEE Trans Inf Forensics Secur 2013;8(11):1876–91. http://dx.doi.org/10.1109/tifs.2013.2279798.
[29] Zalivaka SS, Puchkov AV, Klybik VP, Ivaniuk AA, Chang C-H. 2016 21st Asia and South Pacific design automation conference. IEEE; 2016-01, p. 533–8. http://dx.doi.org/10.1109/aspdac.2016.7428066, URL: http://ieeexplore.ieee.org/document/7428066/.
[30] Becker GT. IEEE Trans Comput-Aided Des Integr Circuits Syst 2015;34(8):1295–307. http://dx.doi.org/10.1109/tcad.2015.2427259.
[31] Shi J, Lu Y, Zhang J. IEEE Trans Comput-Aided Des Integr Circuits Syst 2019;1. http://dx.doi.org/10.1109/tcad.2019.2962115, URL: https://ieeexplore.ieee.org/document/8941137/.
[32] Majzoobi M, Koushanfar F, Devadas S. 2010 IEEE international workshop on information forensics and security. Seattle, WA, USA: IEEE; 2010, p. 1–6. http://dx.doi.org/10.1109/wifs.2010.5711471.
[33] Yang B, Rožic V, Grujic M, Mentens N, Verbauwhede I. IACR Trans Cryptogr Hardw Embed Syst 2018;267–92. http://dx.doi.org/10.46586/tches.v2018.i3.267-292.

[34] Shi Z, Ma C, Cote J, Wang B. In: Tehranipoor M, Wang C, editors. Introduction to hardware security and trust. New York, NY: Springer; 2012, p. 27–50. http://dx.doi.org/10.1007/978-1-4419-8080-9_2.

[35] Good T, Benaissa M. In: Rao JR, Sunar B, editors. Cryptographic hardware and embedded systems. Lecture notes in computer science, Berlin, Heidelberg: Springer; 2005, p. 427–40. http://dx.doi.org/10.1007/11545262_31.

[36] Chen S, Li B, Chen Z, Zhang Y, Wang C, Tao C. IEEE Internet Things J 2021;1. http://dx.doi.org/10.1109/jiot.2021.3065836.

[37] Qureshi MA, Munir A. IEEE Trans Dependable Secure Comput 2021;1. http://dx.doi.org/10.1109/TDSC.2021.3059454.

[38] Ni L, Wang P, Zhang Y, Li G, Ding L, Zhang J. Comput Electr Eng 2023;105:108540. http://dx.doi.org/10.1016/j.compeleceng.2022.108540, URL: https://linkinghub.elsevier.com/retrieve/pii/S0045790622007558.

[39] Zhang J, Ding L, Chen Z, Li W, Qu G. Proceedings of the 59th ACM/IEEE design automation conference. Association for Computing Machinery; 2022-08-23, p. 73–8. http://dx.doi.org/10.1145/3489517.3530412, https://doi.org/10.1145/3489517.3530412.

[40] Sadhu PK, Yanambaka VP. 2022 IEEE computer society annual symposium on VLSI. IEEE; 2022, p. 452–3. http://dx.doi.org/10.1109/ISVLSI54635.2022.00102, URL: https://ieeexplore.ieee.org/document/9912013/.

[41] Idriss H, Rojas P, Alahmadi S, Idriss T, Carlson A, Bayoumi M. 2022 IEEE international symposium on circuits and systems. IEEE; 2022, p. 175–9. http://dx.doi.org/10.1109/ISCAS48785.2022.9937489.

[42] Gope P, Lee J, Quek TQS. IEEE Trans Inf Forensics Secur 2018;13(11):2831–43. http://dx.doi.org/10.1109/tifs.2018.2832849.